

Here is the complete, start-to-finish workflow. You will collect your sensor data using your board, train the AI decision tree model on your PC in **Spyder**, and then deploy the full navigation system back to your **MicroPython device via Thonny**.

Phase 1: Collect Labeled Sensor Data (Thonny)

You need to capture raw feature values (magnitude and variance) from your LSM303DLHC sensor to feed into Spyder.

1. File Details

- **Tool:** Thonny IDE
- **Where to Save:** Save as **data_collector.py** on your **MicroPython Device** (or run directly from PC RAM).

2. Code (data_collector.py)

Python

```
from machine import Pin, I2C
```

```
import utime, math
```

```
i2c = I2C(0, scl=Pin(1), sda=Pin(0), freq=100000)
```

```
ACC_ADDR = 0x19
```

```
# Initialize LSM303DLHC Accelerometer
```

```
i2c.writeto_mem(ACC_ADDR, 0x20, bytes([0x57])) # 100Hz continuous
```

```
i2c.writeto_mem(ACC_ADDR, 0x23, bytes([0x08]))
```

```
def signed_12bit(val):
```

```
    val >>= 4
```

```
    return val - 4096 if val >= 2048 else val
```

```
mag_window = []
```

```
prev_x, prev_y, prev_z = None, None, None
```

```
print("=== SENSOR DATA COLLECTOR STARTED ===")
```

```
print("Copy the printed (magnitude, variance) values below:")
```

```
print("-----")
```

while True:

```
data = i2c.readfrom_mem(ACC_ADDR, 0x28 | 0x80, 6)
```

```
if data:
```

```
    x = signed_12bit((data[1] << 8) | data[0])
```

```
    y = signed_12bit((data[3] << 8) | data[2])
```

```
    z = signed_12bit((data[5] << 8) | data[4])
```

```
if prev_x is None:
```

```
    prev_x, prev_y, prev_z = x, y, z
```

```
    continue
```

```
dx, dy, dz = x - prev_x, y - prev_y, z - prev_z
```

```
prev_x, prev_y, prev_z = x, y, z
```

```
raw_mag = math.sqrt(dx*dx + dy*dy + dz*dz)
```

```
mag_window.append(raw_mag)
```

```
if len(mag_window) > 10:
```

```
    mag_window.pop(0)
```

```
mean_mag = sum(mag_window) / len(mag_window)
```

```
variance = sum((m - mean_mag)**2 for m in mag_window) / len(mag_window)
```

```
# Prints output to copy into Spyder
```

```
print(f"[{round(mean_mag, 2)}, {round(variance, 2)}],")
```

```
utime.sleep_ms(50)
```

3. How to Collect the Data

1. Run data_collector.py while keeping the board flat and motionless on a table for 10 seconds. Copy the printed lines from the Thonny Shell.
2. Run it again while shaking the board rapidly in your hand. Copy those values.
3. Run it a third time while taking steps / pushing the board in straight lines. Copy those values.

Phase 2: Train AI & Generate MicroPython Code (Spyder)

You will input your collected values into **Spyder**, train a DecisionTreeClassifier, and run a converter function that **automatically generates the exact MicroPython code** for your device.

1. File Details

- **Tool:** Spyder IDE
- **Where to Save:** Save as **train_and_export.py** on your **Computer / PC**.

2. Code (train_and_export.py)

Python

```
import numpy as np
```

```
from sklearn.tree import DecisionTreeClassifier, _tree
```

```
# =====
```

```
# 1. WHERE TO CHANGE LABELS AND DATA
```

```
# =====
```

```
# Map numeric IDs to human-readable string labels
```

```
CLASS_NAMES = {0: "SITTING", 1: "SHAKING", 2: "WALKING"}
```

```
# PASTE YOUR COLLECTED [magnitude, variance] READINGS HERE:
```

```
# Label 0 = SITTING data
```

```
# Label 1 = SHAKING data
```

```
# Label 2 = WALKING data
```

```
sitting_data = [
```

```
    [2.1, 0.4], [3.5, 0.8], [1.2, 0.2], [4.0, 1.1], [2.8, 0.5]
```

```
]
```

```
shaking_data = [
```

```
    [85.4, 420.5], [92.1, 510.2], [110.0, 630.0], [75.2, 380.0], [98.5, 490.1]
```

```
]
```

```
walking_data = [
```

```

[35.2, 115.4], [42.6, 140.1], [38.0, 125.0], [45.1, 150.2], [40.3, 132.8]
]

# Build Feature Matrix (X) and Target Labels (y)
X_train = np.array(sitting_data + shaking_data + walking_data)
y_train = np.array(
    [0]*len(sitting_data) +
    [1]*len(shaking_data) +
    [2]*len(walking_data)
)

# =====
# 2. TRAIN DECISION TREE CLASSIFIER
# =====

clf = DecisionTreeClassifier(max_depth=3, random_state=42)
clf.fit(X_train, y_train)

# =====
# 3. AUTOMATIC CONVERTER: DECISION TREE -> MICROPYTHON CODE
# =====

def export_micropython_code(tree, feature_names):
    tree_ = tree.tree_
    feature_name = [
        feature_names[i] if i != _tree.TREE_UNDEFINED else "undefined!"
        for i in tree_.feature
    ]

    code = "def predict_activity_ai(magnitude, variance):\n"

    def recurse(node, depth):
        nonlocal code

```

```

indent = "  " * depth

if tree_.feature[node] != _tree.TREE_UNDEFINED:
    name = feature_name[node]
    threshold = round(tree_.threshold[node], 2)
    code += f'{indent}if {name} <= {threshold}:\n'
    recurse(tree_.children_left[node], depth + 1)
    code += f'{indent}else:\n'
    recurse(tree_.children_right[node], depth + 1)
else:
    class_id = int(np.argmax(tree_.value[node]))
    class_label = CLASS_NAMES.get(class_id, "UNKNOWN")
    code += f'{indent}return "{class_label}"\n'

recurse(0, 1)

return code

```

Generate MicroPython Function

```
micropython_code = export_micropython_code(clf, ["magnitude", "variance"])
```

```

print("\n=====")
print(" GENERATED MICROPYTHON CODE (COPY THIS TO THONNY)")
print("=====")
print(micropython_code)
print("=====")

```

3. How to Change Labels in Spyder

If you want to add a new state (e.g., RUNNING as 3):

1. Add 3: "RUNNING" inside CLASS_NAMES.
2. Create a list running_data = [[...], [...]].
3. Append running_data to X_train and add [3]*len(running_data) to y_train.
4. Click **Run** in Spyder. The console will print your generated predict_activity_ai MicroPython function!

Phase 3: Final Deployment on Device (Thonny)

Combine the Spyder-generated AI function with the full PDR dead-reckoning navigation system and flash it to your RT1064 EVK board.

1. File Details

- **Tool:** Thonny IDE
- **Where to Save:** Click **File** → **Save As...** → Choose **MicroPython Device** → Save strictly as **main.py**.

2. Code (main.py)

Python

```
from machine import Pin, I2C
```

```
import utime
```

```
import math
```

```
# =====
```

```
# HARDWARE CONFIGURATION
```

```
# =====
```

```
I2C_ID = 0
```

```
SDA_PIN = 0
```

```
SCL_PIN = 1
```

```
I2C_FREQ = 100000
```

```
ACC_ADDRESS = 0x19
```

```
MAG_ADDRESS = 0x1E
```

```
CTRL_REG1_A = 0x20
```

```
CTRL_REG4_A = 0x23
```

```
OUT_X_L_A = 0x28
```

```
AUTO_INCREMENT = 0x80
```

```
CRA_REG_M = 0x00
```

```
CRB_REG_M = 0x01
```

```
MR_REG_M = 0x02
```

```
OUT_X_H_M = 0x03
```

```
# Tuning & Navigation Parameters
```

```
STRIDE_LENGTH_M = 0.75    # Distance per step (meters)
```

```
STEP_COOLDOWN_MS = 400    # Recoil lock window
```

```
ALPHA = 0.20              # EMA filter smoothing factor
```

```
ORIGIN_RADIUS_M = 1.5     # Return to start threshold (meters)
```

```
# System State Variables
```

```
x_bias, y_bias, z_bias = 0, 0, 0
```

```
mag_x_bias, mag_y_bias = 0, 0
```

```
prev_x, prev_y, prev_z = None, None, None
```

```
filtered_mag = None
```

```
total_steps = 0
```

```
total_distance = 0.0
```

```
last_step_time = 0
```

```
prev_activity = None
```

```
pos_x = 0.0
```

```
pos_y = 0.0
```

```
current_leg_dir = None
```

```
current_leg_dist = 0.0
```

```
route_history = []
```

```
mag_window = []
```

```
i2c = I2C(I2C_ID, scl=Pin(SCL_PIN), sda=Pin(SDA_PIN), freq=I2C_FREQ)
```

```
# =====
```

```
# LOW-LEVEL DRIVERS
```

```
# =====
```

```

def write_reg(addr, reg, value):
    try:
        i2c.writeto_mem(addr, reg, bytes([value]))
        return True
    except OSError:
        return False

def read_regs(addr, reg, length):
    try:
        return i2c.readfrom_mem(addr, reg, length)
    except OSError:
        return None

def signed_12bit(val):
    val >>= 4
    if val >= 2048:
        val -= 4096
    return val

def signed_16bit(val):
    if val >= 32768:
        val -= 65536
    return val

def init_lsm303dlhc():
    write_reg(ACC_ADDRESS, CTRL_REG1_A, 0x57)
    write_reg(ACC_ADDRESS, CTRL_REG4_A, 0x08)
    write_reg(MAG_ADDRESS, CRA_REG_M, 0x14)
    write_reg(MAG_ADDRESS, CRB_REG_M, 0x20)
    write_reg(MAG_ADDRESS, MR_REG_M, 0x00)
    utime.sleep_ms(100)

```



```

def read_accel():
    data = read_regs(ACC_ADDRESS, OUT_X_L_A | AUTO_INCREMENT, 6)
    if data is None: return None
    x = signed_12bit((data[1] << 8) | data[0])
    y = signed_12bit((data[3] << 8) | data[2])
    z = signed_12bit((data[5] << 8) | data[4])
    return x, y, z

```

```

def read_mag():
    data = read_regs(MAG_ADDRESS, OUT_X_H_M, 6)
    if data is None: return None
    mx = signed_16bit((data[0] << 8) | data[1])
    mz = signed_16bit((data[2] << 8) | data[3])
    my = signed_16bit((data[4] << 8) | data[5])
    return mx, my, mz

```

```

def calibrate_sensors():
    global x_bias, y_bias, z_bias, mag_x_bias, mag_y_bias

    print("Step 1/2: Calibrating Accelerometer (KEEP STILL)...")
    sx, sy, sz = 0, 0, 0
    valid = 0
    while valid < 100:
        vals = read_accel()
        if vals:
            sx += vals[0]
            sy += vals[1]
            sz += vals[2]
            valid += 1
        utime.sleep_ms(10)

```

```
x_bias = sx / 100
```

```
y_bias = sy / 100
```

```
z_bias = (sz / 100) - 1024
```

```
print("Step 2/2: Calibrating Compass (ROTATE SENSOR IN AIR FOR 4 SEC)...")
```

```
min_x, max_x = 32767, -32768
```

```
min_y, max_y = 32767, -32768
```

```
start_time = utime.ticks_ms()
```

```
while utime.ticks_diff(utime.ticks_ms(), start_time) < 4000:
```

```
    m = read_mag()
```

```
    if m:
```

```
        if m[0] < min_x: min_x = m[0]
```

```
        if m[0] > max_x: max_x = m[0]
```

```
        if m[1] < min_y: min_y = m[1]
```

```
        if m[1] > max_y: max_y = m[1]
```

```
    utime.sleep_ms(20)
```

```
mag_x_bias = (max_x + min_x) / 2
```

```
mag_y_bias = (max_y + min_y) / 2
```

```
print("Calibration Complete!")
```

```
# =====
```

```
# PASTE YOUR SPYDER-GENERATED FUNCTION HERE
```

```
# =====
```

```
def predict_activity_ai(magnitude, variance):
```

```
    if magnitude <= 18.6:
```

```
        return "SITTING"
```

```
    else:
```

```
        if variance > 275.5:
```

```
            return "SHAKING"
```

```

else:
    return "WALKING"

def calculate_heading(mx, my):
    mx_corr = mx - mag_x_bias
    my_corr = my - mag_y_bias
    heading_deg = math.degrees(math.atan2(my_corr, mx_corr))
    if heading_deg < 0:
        heading_deg += 360.0
    return heading_deg

def get_cardinal_direction(heading):
    if (heading >= 337.5) or (heading < 22.5): return "NORTH"
    elif 22.5 <= heading < 67.5: return "NORTH-EAST"
    elif 67.5 <= heading < 112.5: return "EAST"
    elif 112.5 <= heading < 157.5: return "SOUTH-EAST"
    elif 157.5 <= heading < 202.5: return "SOUTH"
    elif 202.5 <= heading < 247.5: return "SOUTH-WEST"
    elif 247.5 <= heading < 292.5: return "WEST"
    elif 292.5 <= heading < 337.5: return "NORTH-WEST"
    return "UNKNOWN"

# =====
# MAIN APPLICATION LOOP
# =====

def main():
    global prev_x, prev_y, prev_z, filtered_mag, total_steps, total_distance, last_step_time,
    prev_activity

    global pos_x, pos_y, current_leg_dir, current_leg_dist

    print("-----")

```

```
print("RT1064 PDR Navigation & Route Tracker")
print("-----")
```

```
init_lsm303dlhc()
calibrate_sensors()
```

```
print("\nInitial Position Set: (X: 0.0m, Y: 0.0m)")
print("Waiting for motion...")
print("-----")
```

```
while True:
```

```
    try:
```

```
        acc_vals = read_accel()
```

```
        mag_vals = read_mag()
```

```
        if acc_vals is None or mag_vals is None:
```

```
            continue
```

```
        x = acc_vals[0] - x_bias
```

```
        y = acc_vals[1] - y_bias
```

```
        z = acc_vals[2] - z_bias
```

```
        if prev_x is None:
```

```
            prev_x, prev_y, prev_z = x, y, z
```

```
            continue
```

```
        dx = x - prev_x
```

```
        dy = y - prev_y
```

```
        dz = z - prev_z
```

```
        prev_x, prev_y, prev_z = x, y, z
```

```
raw_mag = math.sqrt(dx * dx + dy * dy + dz * dz)
```

```
if filtered_mag is None:
```

```
    filtered_mag = raw_mag
```

```
else:
```

```
    filtered_mag = (ALPHA * raw_mag) + ((1 - ALPHA) * filtered_mag)
```

```
mag_window.append(filtered_mag)
```

```
if len(mag_window) > 10:
```

```
    mag_window.pop(0)
```

```
mean_mag = sum(mag_window) / len(mag_window)
```

```
variance = sum((m - mean_mag) ** 2 for m in mag_window) / len(mag_window)
```

```
current_time = utime.ticks_ms()
```

```
time_diff = utime.ticks_diff(current_time, last_step_time)
```

```
activity = predict_activity_ai(filtered_mag, variance)
```

```
heading = calculate_heading(mag_vals[0], mag_vals[1])
```

```
cardinal_dir = get_cardinal_direction(heading)
```

```
# CONDITION 1: WALKING (Updates Position & Route)
```

```
if activity == "WALKING":
```

```
    if time_diff > STEP_COOLDOWN_MS:
```

```
        last_step_time = current_time
```

```
        total_steps += 1
```

```
        total_distance += STRIDE_LENGTH_M
```

```
        rad = math.radians(heading)
```

```
        pos_x += STRIDE_LENGTH_M * math.sin(rad)
```

```

pos_y += STRIDE_LENGTH_M * math.cos(rad)

if current_leg_dir is None:
    current_leg_dir = cardinal_dir
    current_leg_dist = STRIDE_LENGTH_M
elif cardinal_dir == current_leg_dir:
    current_leg_dist += STRIDE_LENGTH_M
else:
    route_history.append((current_leg_dir, current_leg_dist))

    print(">>> TURN DETECTED <<<")

    print(f"Completed Leg : Traveled {round(current_leg_dist, 2)}m towards
{current_leg_dir}")

    print(f"New Direction : Turning towards {cardinal_dir} ({round(heading, 1)}°)")
    print("-----")
    current_leg_dir = cardinal_dir
    current_leg_dist = STRIDE_LENGTH_M

print(">>> STEP DETECTED <<<")
print("Activity    : WALKING")
print("Step Count   :", total_steps)
print("Total Dist   :", round(total_distance, 2), "m")
print("Heading      :", cardinal_dir, f"({round(heading, 1)}°)")
print(f"Coordinates   : X = {round(pos_x, 2)}m, Y = {round(pos_y, 2)}m")

# Loop Closure Check
dist_to_start = math.sqrt(pos_x * pos_x + pos_y * pos_y)
if total_steps > 8 and dist_to_start <= ORIGIN_RADIUS_M:
    print("\n=====")
    print(">>> RETURNED TO INITIAL POINT <<<")
    print(f"Distance from Start: {round(dist_to_start, 2)}m")
    print("ROUTE SUMMARY:")

```

```

temp_history = route_history + [(current_leg_dir, current_leg_dist)]
for idx, (leg_dir, leg_dist) in enumerate(temp_history, 1):
    print(f" Leg {idx}: Walked {round(leg_dist, 2)}m towards {leg_dir}")
print(f"Total Distance: {round(total_distance, 2)}m")
print("=====\n")

print("-----")
prev_activity = "WALKING"

# CONDITION 2: SHAKING
elif activity == "SHAKING":
    if prev_activity != "SHAKING":
        print(">>> ACTION DETECTED <<<")
        print("Activity    : SHAKING (Step counter frozen)")
        print("-----")
        prev_activity = "SHAKING"

# CONDITION 3: SITTING
elif activity == "SITTING":
    if prev_activity != "SITTING" and prev_activity is not None:
        print(">>> ACTION DETECTED <<<")
        print("Activity    : SITTING (Step counter frozen)")
        print("-----")
        prev_activity = "SITTING"

    utime.sleep_ms(30)

except KeyboardInterrupt:
    print("\nProgram Terminated.")
    break
except Exception as e:

```

```

        print("Error:", e)

        utime.sleep_ms(200)

if __name__ == "__main__":
    main()

```

Summary Cheat-Sheet

Phase	Script Name	Software	Save Location	Primary Role
Phase 1	data_collector.py	Thonny	MicroPython Device	Logs raw [magnitude, variance]
Phase 2	train_and_export.py	Spyder	PC Local Folder	Trains Decision Tree & prints
Phase 3	main.py	Thonny	MicroPython Device	Runs real-time AI classification

Yes, you paste the values into the python lists manually—but it is super quick!

Because of how we formatted data_collector.py in Phase 1, the terminal prints the data with brackets and commas already added (e.g., [2.1, 0.4],).

You can literally just **copy a whole block of lines from the Thonny terminal** and **paste them directly into Spyder**.

How to Add More Readings in train_and_export.py

In your Spyder code, simply paste as many [magnitude, variance] rows into each list as you want. There is no limit—the more readings you add, the smarter your decision tree becomes!

Here is how your lists in Spyder should look when you paste multiple readings:

Python

```

# =====
# PASTE YOUR COLLECTED [magnitude, variance] READINGS HERE:
# =====

```

```

sitting_data = [
    # --- Paste your sitting lines from Thonny terminal here ---
    [2.1, 0.4],

```



```
[3.5, 0.8],  
[1.2, 0.2],  
[4.0, 1.1],  
[2.8, 0.5],  
[1.9, 0.3], # Added more readings...  
[3.1, 0.6],  
[2.4, 0.4]  
]
```

```
shaking_data = [  
    # --- Paste your shaking lines from Thonny terminal here ---  
    [85.4, 420.5],  
    [92.1, 510.2],  
    [110.0, 630.0],  
    [75.2, 380.0],  
    [98.5, 490.1],  
    [105.2, 550.8], # Added more readings...  
    [88.9, 440.3]  
]
```

```
walking_data = [  
    # --- Paste your walking lines from Thonny terminal here ---  
    [35.2, 115.4],  
    [42.6, 140.1],  
    [38.0, 125.0],  
    [45.1, 150.2],  
    [40.3, 132.8],  
    [39.8, 128.4], # Added more readings...  
    [41.5, 135.0]  
]
```

Pro-Tip for Pasting

- Make sure every row inside the square brackets has a comma at the end , except optionally the very last one.
- Aim for at least **15 to 30 lines** for each motion state to get reliable decision rules!

You said

we can add any number of values??

Gemini said

Yes, absolutely! You can add as many values as you want—whether it is 20 rows or 2,000 rows per activity.

In fact, **more data usually makes the AI model smarter and more accurate**, because it helps the Decision Tree find the true boundary between sitting, shaking, and walking instead of guessing based on just a few points.

Tips for Adding Large Datasets

1. **Keep it Balanced:** Try to collect roughly the same amount of data for each activity (e.g., ~50 lines for sitting, ~50 lines for shaking, and ~50 lines for walking).
2. **Include Variety:**
 - For **sitting**: collect data while the board is flat on a desk and while holding it in your hand.
 - For **walking**: collect data while walking slow, walking fast, and pushing the sensor horizontally.
3. **Pasting Format:** Just copy the lines directly from your Thonny terminal and paste them inside the square brackets [...]. As long as each pair is wrapped in brackets and separated by a comma, Python in Spyder will handle all of them instantly!

Yes, for this specific project, pasting python lists directly into Spyder is actually a much better and faster idea than using a CSV file.

Here is why this method works so well for your setup:

Why Python Lists Beat CSV Files Here

1. **Zero File I/O Friction:** Saving to a CSV file on a MicroPython board requires mounting storage, handling file streams (`open('data.csv', 'w')`), and transferring files back to your PC. Copying text directly from the Thonny Shell bypasses file transfer entirely.
2. **Instant Spyder Execution:** When you paste your lists directly into `train_and_export.py`, you don't need pandas or file-loading paths (`pd.read_csv('...')`). You just press **Run** in Spyder, and it trains instantly.
3. **Fewer Technical Failure Points:** MicroPython file systems can occasionally corrupt or overflow flash memory if files aren't closed properly during data logging. Terminal prints avoid this risk completely.

When Would a CSV File Be Better?

A CSV file is only needed if you are collecting **massive datasets** (e.g., thousands of rows collected over hours/days) or training complex deep-learning models (like Neural Networks).

Since your Decision Tree only needs about **30 to 100 samples per activity** to find clean thresholds (magnitude and variance), copying and pasting directly into python lists is the cleanest, simplest, and most efficient approach!

2nd

Here is a breakdown of what the **Complete Trajectory and Route History Summary** looks like when printed in your terminal, followed by the **production-ready MicroPython code (main.py)**.

What the AI/ML Output Looks Like

When you walk a route (for example: walking North →turning East →turning South →returning to Start), the AI/ML pipeline processes every stride, filters out noise, detects turns, and prints this **Route Summary** when you re-enter the starting zone:

Plaintext

```
=====
```

```
>>> AI-VERIFIED ROUTE & TRAJECTORY SUMMARY <<<
```

```
=====
```

```
Status      : LOOP CLOSURE CONFIRMED (Returned to Origin)
```

```
Dist to Start : 0.82m (Within 1.5m Radius)
```

```
Total Steps  : 20
```

```
Total Distance: 15.00m
```

SEGMENT HISTORY LOG:

```
Leg 1 : Walked 3.75m towards NORTH (Heading: 3.2°)
```

```
Leg 2 : Walked 3.75m towards EAST  (Heading: 88.7°)
```

```
Leg 3 : Walked 3.75m towards SOUTH (Heading: 178.4°)
```

```
Leg 4 : Walked 3.75m towards WEST  (Heading: 268.9°)
```

FINAL BOUNDING COORDINATES:

```
Current X : 0.12m
```

```
Current Y : 0.05m
```

=====

Complete Production Code (main.py)

This clean implementation incorporates the **low-pass EMA filter**, **Spyder-trained Decision Tree AI gating**, **LSM303DLHC hard/soft iron calibration**, **turn detection**, and **loop-closure route summary**.

Save this directly as **main.py** on your MicroPython device in Thonny:

Python

```
from machine import Pin, I2C
```

```
import utime
```

```
import math
```

```
# =====
```

```
# 1. HARDWARE & SYSTEM CONFIGURATION
```

```
# =====
```

```
I2C_ID = 0
```

```
SDA_PIN = 0
```

```
SCL_PIN = 1
```

```
I2C_FREQ = 100000
```

```
ACC_ADDRESS = 0x19
```

```
MAG_ADDRESS = 0x1E
```

```
CTRL_REG1_A = 0x20
```

```
CTRL_REG4_A = 0x23
```

```
OUT_X_L_A = 0x28
```

```
AUTO_INCREMENT = 0x80
```

```
CRA_REG_M = 0x00
```

```
CRB_REG_M = 0x01
```

```
MR_REG_M = 0x02
```

```
OUT_X_H_M = 0x03
```

```
# Tuning & Navigation Parameters

STRIDE_LENGTH_M = 0.75    # Distance per step (meters)

STEP_COOLDOWN_MS = 400    # Recoil lockout window

ALPHA = 0.20              # EMA filter smoothing factor

ORIGIN_RADIUS_M = 1.5     # Return to origin loop-closure radius (meters)
```

```
# Global State Registers

x_bias, y_bias, z_bias = 0, 0, 0

mag_x_bias, mag_y_bias = 0, 0
```

```
prev_x, prev_y, prev_z = None, None, None

filtered_mag = None

total_steps = 0

total_distance = 0.0

last_step_time = 0

prev_activity = None
```

```
pos_x = 0.0

pos_y = 0.0

current_leg_dir = None

current_leg_dist = 0.0

route_history = []

mag_window = []
```

```
i2c = I2C(I2C_ID, scl=Pin(SCL_PIN), sda=Pin(SDA_PIN), freq=I2C_FREQ)
```

```
# =====
```

```
# 2. I2C COMMUNICATION & SENSOR DRIVERS
```

```
# =====
```

```
def write_reg(addr, reg, value):
```

```
    try:
```

```

        i2c.writeto_mem(addr, reg, bytes([value]))

    return True

except OSError:

    return False


def read_regs(addr, reg, length):

    try:

        return i2c.readfrom_mem(addr, reg, length)

    except OSError:

        return None


def signed_12bit(val):

    val >>= 4

    if val >= 2048:

        val -= 4096

    return val


def signed_16bit(val):

    if val >= 32768:

        val -= 65536

    return val


def init_lsm303dlhc():

    write_reg(ACC_ADDRESS, CTRL_REG1_A, 0x57) # 100Hz continuous mode
    write_reg(ACC_ADDRESS, CTRL_REG4_A, 0x08) # High resolution mode
    write_reg(MAG_ADDRESS, CRA_REG_M, 0x14) # 15Hz magnetometer update
    write_reg(MAG_ADDRESS, CRB_REG_M, 0x20) # Sensor gain setup
    write_reg(MAG_ADDRESS, MR_REG_M, 0x00) # Continuous conversion
    utime.sleep_ms(100)


def read_accel():

```

```

data = read_regs(ACC_ADDRESS, OUT_X_L_A | AUTO_INCREMENT, 6)
if data is None: return None
x = signed_12bit((data[1] << 8) | data[0])
y = signed_12bit((data[3] << 8) | data[2])
z = signed_12bit((data[5] << 8) | data[4])
return x, y, z

```

```

def read_mag():
    data = read_regs(MAG_ADDRESS, OUT_X_H_M, 6)
    if data is None: return None
    mx = signed_16bit((data[0] << 8) | data[1])
    mz = signed_16bit((data[2] << 8) | data[3])
    my = signed_16bit((data[4] << 8) | data[5])
    return mx, my, mz

```

```

def calibrate_sensors():
    global x_bias, y_bias, z_bias, mag_x_bias, mag_y_bias

    print("\n--- SENSOR CALIBRATION PROCEDURE ---")
    print("Step 1/2: Calibrating Accelerometer (KEEP SENSOR STILL)...")
    sx, sy, sz = 0, 0, 0
    valid = 0
    while valid < 100:
        vals = read_accel()
        if vals:
            sx += vals[0]
            sy += vals[1]
            sz += vals[2]
            valid += 1
            utime.sleep_ms(10)
    x_bias = sx / 100

```

```
y_bias = sy / 100
```

```
z_bias = (sz / 100) - 1024
```

```
print("Step 2/2: Calibrating Magnetometer (ROTATE IN AIR FOR 4 SEC)...")
```

```
min_x, max_x = 32767, -32768
```

```
min_y, max_y = 32767, -32768
```

```
start_time = utime.ticks_ms()
```

```
while utime.ticks_diff(utime.ticks_ms(), start_time) < 4000:
```

```
    m = read_mag()
```

```
    if m:
```

```
        if m[0] < min_x: min_x = m[0]
```

```
        if m[0] > max_x: max_x = m[0]
```

```
        if m[1] < min_y: min_y = m[1]
```

```
        if m[1] > max_y: max_y = m[1]
```

```
    utime.sleep_ms(20)
```

```
mag_x_bias = (max_x + min_x) / 2
```

```
mag_y_bias = (max_y + min_y) / 2
```

```
print("Calibration Completed Successfully!\n")
```

```
# =====
```

```
# 3. AI CLASSIFIER & HEADING ALGORITHMS
```

```
# =====
```

```
def predict_activity_ai(magnitude, variance):
```

```
    """
```

```
    AI Decision Tree classifier trained in Spyder.
```

```
    Prevents false steps during SITTING or SHAKING motion.
```

```
    """
```

```
    if magnitude <= 18.6:
```

```
        return "SITTING"
```



```

else:
    if variance > 275.5:
        return "SHAKING"
    else:
        return "WALKING"

```

```

def calculate_heading(mx, my):
    mx_corr = mx - mag_x_bias
    my_corr = my - mag_y_bias
    heading_deg = math.degrees(math.atan2(my_corr, mx_corr))
    if heading_deg < 0:
        heading_deg += 360.0
    return heading_deg

```

```

def get_cardinal_direction(heading):
    if (heading >= 337.5) or (heading < 22.5): return "NORTH"
    elif 22.5 <= heading < 67.5: return "NORTH-EAST"
    elif 67.5 <= heading < 112.5: return "EAST"
    elif 112.5 <= heading < 157.5: return "SOUTH-EAST"
    elif 157.5 <= heading < 202.5: return "SOUTH"
    elif 202.5 <= heading < 247.5: return "SOUTH-WEST"
    elif 247.5 <= heading < 292.5: return "WEST"
    elif 292.5 <= heading < 337.5: return "NORTH-WEST"
    return "UNKNOWN"

```

```

# =====

```

```

# 4. MAIN NAVIGATION & ROUTE TRACKING LOOP

```

```

# =====

```

```

def main():
    global prev_x, prev_y, prev_z, filtered_mag, total_steps, total_distance, last_step_time,
    prev_activity

```

```
global pos_x, pos_y, current_leg_dir, current_leg_dist
```

```
init_lsm303dlhc()
```

```
calibrate_sensors()
```

```
print("=====")
```

```
print(" RT1064 AI-PDR SENSOR TRACKING ENGINE ONLINE")
```

```
print("=====")
```

```
print("Initial Position : X = 0.00m, Y = 0.00m (Origin)")
```

```
print("Waiting for motion...")
```

```
print("-----")
```

```
while True:
```

```
    try:
```

```
        acc_vals = read_accel()
```

```
        mag_vals = read_mag()
```

```
        if acc_vals is None or mag_vals is None:
```

```
            continue
```

```
        x = acc_vals[0] - x_bias
```

```
        y = acc_vals[1] - y_bias
```

```
        z = acc_vals[2] - z_bias
```

```
        if prev_x is None:
```

```
            prev_x, prev_y, prev_z = x, y, z
```

```
            continue
```

```
        dx = x - prev_x
```

```
        dy = y - prev_y
```

```
        dz = z - prev_z
```

```

prev_x, prev_y, prev_z = x, y, z

raw_mag = math.sqrt(dx*dx + dy*dy + dz*dz)

# Apply Low-Pass EMA Smoothing Filter
if filtered_mag is None:
    filtered_mag = raw_mag
else:
    filtered_mag = (ALPHA * raw_mag) + ((1 - ALPHA) * filtered_mag)

# Maintain 10-sample window for variance extraction
mag_window.append(filtered_mag)
if len(mag_window) > 10:
    mag_window.pop(0)

mean_mag = sum(mag_window) / len(mag_window)
variance = sum((m - mean_mag)**2 for m in mag_window) / len(mag_window)

current_time = utime.ticks_ms()
time_diff = utime.ticks_diff(current_time, last_step_time)

# AI State Prediction
activity = predict_activity_ai(filtered_mag, variance)

heading = calculate_heading(mag_vals[0], mag_vals[1])
cardinal_dir = get_cardinal_direction(heading)

# --- WALKING STATE: PROCESS STEP & POSITION ---
if activity == "WALKING":
    if time_diff > STEP_COOLDOWN_MS:
        last_step_time = current_time

```

```

total_steps += 1

total_distance += STRIDE_LENGTH_M


# Calculate 2D Coordinate Update
rad = math.radians(heading)
pos_x += STRIDE_LENGTH_M * math.sin(rad)
pos_y += STRIDE_LENGTH_M * math.cos(rad)


# Turn Detection Logic
if current_leg_dir is None:
    current_leg_dir = cardinal_dir
    current_leg_dist = STRIDE_LENGTH_M
elif cardinal_dir == current_leg_dir:
    current_leg_dist += STRIDE_LENGTH_M
else:
    route_history.append((current_leg_dir, current_leg_dist, heading))
    print(">>> TURN DETECTED <<<")
    print(f"Completed Leg : Walked {round(current_leg_dist, 2)}m towards
{current_leg_dir}")
    print(f"New Direction : Heading towards {cardinal_dir} ({round(heading, 1)}°)")
    print("-----")
    current_leg_dir = cardinal_dir
    current_leg_dist = STRIDE_LENGTH_M


print(">>> AI STEP CONFIRMED <<<")
print("Activity    : WALKING")
print("Step Count   :", total_steps)
print("Total Dist   :", round(total_distance, 2), "m")
print("Heading      :", cardinal_dir, f"({round(heading, 1)}°)")
print(f"Coordinates  : X = {round(pos_x, 2)}m, Y = {round(pos_y, 2)}m")

```

```

# Loop Closure Check

dist_to_start = math.sqrt(pos_x*pos_x + pos_y*pos_y)

if total_steps > 8 and dist_to_start <= ORIGIN_RADIUS_M:

    print("\n=====")
    print(">>> AI-VERIFIED ROUTE & TRAJECTORY SUMMARY <<<")
    print("=====")
    print("Status      : LOOP CLOSURE CONFIRMED (Returned to Origin)")
    print(f"Dist to Start : {round(dist_to_start, 2)}m (Within {ORIGIN_RADIUS_M}m
Radius)")
    print(f"Total Steps  : {total_steps}")
    print(f"Total Distance: {round(total_distance, 2)}m\n")
    print("SEGMENT HISTORY LOG:")

    temp_history = route_history + [(current_leg_dir, current_leg_dist, heading)]
    for idx, (leg_dir, leg_dist, leg_head) in enumerate(temp_history, 1):
        print(f" Leg {idx} : Walked {round(leg_dist, 2)}m towards {leg_dir} (Heading:
{round(leg_head, 1)}°)")

    print("\nFINAL BOUNDING COORDINATES:")
    print(f" Current X : {round(pos_x, 2)}m")
    print(f" Current Y : {round(pos_y, 2)}m")
    print("=====\n")

    print("-----")
    prev_activity = "WALKING"

# --- SHAKING STATE: LOCK STEP COUNTER ---

elif activity == "SHAKING":

    if prev_activity != "SHAKING":
        print(">>> MOTION STATE CHANGE <<<")
        print("Activity      : SHAKING (Step Counter Frozen)")
        print("-----")

```

```

        prev_activity = "SHAKING"

# --- SITTING STATE: LOCK STEP COUNTER ---
elif activity == "SITTING":
    if prev_activity != "SITTING" and prev_activity is not None:
        print(">>> MOTION STATE CHANGE <<<")
        print("Activity    : SITTING (Step Counter Frozen)")
        print("-----")
        prev_activity = "SITTING"

    utime.sleep_ms(30)

except KeyboardInterrupt:
    print("\nTracking Stopped by User.")
    break

except Exception as e:
    print("Execution Error:", e)
    utime.sleep_ms(200)

if __name__ == "__main__":
    main()

```

next one

Here is what the live terminal output will look like as you move the sensor around in real time — functioning just like a **GPS Navigation Unit** that updates instantly with every single movement!

Live Real-Time Navigation Display

When you move, turn, or stop, the system outputs clear real-time telemetry updates:

Plaintext

=====

LIVE PDR SENSOR GPS TELEMETRY

=====

Current State : WALKING

Total Distance: 3.75 m | Total Steps: 5

Heading : 88.4° (EAST)

Coordinates : X = +3.75 m, Y = +0.10 m

Vector Motion : Moving EAST at 0.75 m/step

Dist to Start : 3.75 m

=====

Event Notifications (Turns & State Changes)

1. When You Make a Turn (e.g., Turning North to East)

Plaintext

>>> TURN DETECTED <<<

Completed Leg : Walked 5.25 m towards NORTH

New Direction : Now moving EAST (Heading: 91.2°)

Coordinates : X = +0.15 m, Y = +5.25 m

2. When You Stop Moving or Shake the Device

Plaintext

>>> ACTION DETECTED <<<

Current State : SITTING / IDLE

GPS Tracking : LOCKED (Distance & Coordinates Paused)

Coordinates : X = +3.75 m, Y = +5.25 m

3. Complete Route & Trajectory Log (When You Complete a Loop)

Plaintext

=====

ROUTE COMPLETED (RETURNED TO ORIGIN)

=====

Status : LOOP CLOSURE CONFIRMED

Total Steps : 24 steps

Total Distance: 18.00 m

Final Position: X = +0.12 m, Y = -0.08 m (Within 1.5m radius)

JOURNEY ROUTE BREAKDOWN:

Leg 1 : Walked 4.50 m towards NORTH

Leg 2 : Walked 4.50 m towards EAST

Leg 3 : Walked 4.50 m towards SOUTH

Leg 4 : Walked 4.50 m towards WEST

=====

Key Information Displayed on Screen

- **Current Motion State:** WALKING, SITTING, or SHAKING
- **Real-Time Coordinates:** Your exact (X, Y) position in meters relative to where you turned on the sensor (0,0)
- **Live Compass Heading:** Numerical angle (0.0° to 359.9°) plus Cardinal Direction (NORTH, EAST, SOUTH-WEST, etc.)
- **Live Odometry:** Total accumulated distance traveled and total verified step count
- **Distance to Origin:** How far away in meters you currently are from where you started

Ready for the complete code to run this real-time GPS tracking display?

Yes

You said

okay am ready give code

Gemini said

Here is the complete, production-ready **main.py** code for your MicroPython device. It acts as an indoor **Pedestrian Dead Reckoning (PDR) GPS tracker**, giving real-time telemetry updates whenever you move, turn, or pause.

Production Code (main.py)

Save this file directly to your board as **main.py** using Thonny:

Python

```
from machine import Pin, I2C
```

```
import utime
```

```
import math
```



```

# =====

# 1. HARDWARE & SYSTEM CONFIGURATION

# =====

I2C_ID = 0

SDA_PIN = 0

SCL_PIN = 1

I2C_FREQ = 100000


ACC_ADDRESS = 0x19

MAG_ADDRESS = 0x1E


CTRL_REG1_A = 0x20

CTRL_REG4_A = 0x23

OUT_X_L_A = 0x28

AUTO_INCREMENT = 0x80


CRA_REG_M = 0x00

CRB_REG_M = 0x01

MR_REG_M = 0x02

OUT_X_H_M = 0x03


# Navigation & Sensor Tuning Parameters

STRIDE_LENGTH_M = 0.75    # Distance per step/push (meters)

STEP_COOLDOWN_MS = 400    # Minimum time between steps (ms)

ALPHA = 0.20              # EMA filter smoothing constant

ORIGIN_RADIUS_M = 1.5     # Return to start threshold (meters)


# System State Variables

x_bias, y_bias, z_bias = 0, 0, 0

mag_x_bias, mag_y_bias = 0, 0

```

```
prev_x, prev_y, prev_z = None, None, None
```

```
filtered_mag = None
```

```
total_steps = 0
```

```
total_distance = 0.0
```

```
last_step_time = 0
```

```
prev_activity = None
```

```
pos_x = 0.0
```

```
pos_y = 0.0
```

```
current_leg_dir = None
```

```
current_leg_dist = 0.0
```

```
route_history = []
```

```
mag_window = []
```

```
i2c = I2C(I2C_ID, scl=Pin(SCL_PIN), sda=Pin(SDA_PIN), freq=I2C_FREQ)
```

```
# =====
```

```
# 2. LOW-LEVEL SENSOR DRIVERS
```

```
# =====
```

```
def write_reg(addr, reg, value):
```

```
    try:
```

```
        i2c.writeto_mem(addr, reg, bytes([value]))
```

```
        return True
```

```
    except OSError:
```

```
        return False
```

```
def read_regs(addr, reg, length):
```

```
    try:
```

```
        return i2c.readfrom_mem(addr, reg, length)
```

```
    except OSError:
```

```
    return None
```

```
def signed_12bit(val):
```

```
    val >>= 4
```

```
    if val >= 2048:
```

```
        val -= 4096
```

```
    return val
```

```
def signed_16bit(val):
```

```
    if val >= 32768:
```

```
        val -= 65536
```

```
    return val
```

```
def init_lsm303dlhc():
```

```
    write_reg(ACC_ADDRESS, CTRL_REG1_A, 0x57) # 100Hz accelerometer
```

```
    write_reg(ACC_ADDRESS, CTRL_REG4_A, 0x08) # High resolution
```

```
    write_reg(MAG_ADDRESS, CRA_REG_M, 0x14) # 15Hz magnetometer
```

```
    write_reg(MAG_ADDRESS, CRB_REG_M, 0x20) # Gain setup
```

```
    write_reg(MAG_ADDRESS, MR_REG_M, 0x00) # Continuous conversion
```

```
    utime.sleep_ms(100)
```

```
def read_accel():
```

```
    data = read_regs(ACC_ADDRESS, OUT_X_L_A | AUTO_INCREMENT, 6)
```

```
    if data is None: return None
```

```
    x = signed_12bit((data[1] << 8) | data[0])
```

```
    y = signed_12bit((data[3] << 8) | data[2])
```

```
    z = signed_12bit((data[5] << 8) | data[4])
```

```
    return x, y, z
```

```
def read_mag():
```

```
    data = read_regs(MAG_ADDRESS, OUT_X_H_M, 6)
```

```

if data is None: return None

mx = signed_16bit((data[0] << 8) | data[1])
mz = signed_16bit((data[2] << 8) | data[3])
my = signed_16bit((data[4] << 8) | data[5])

return mx, my, mz

```

```
def calibrate_sensors():
```

```
    global x_bias, y_bias, z_bias, mag_x_bias, mag_y_bias
```

```
    print("\n-----")
```

```
    print("    SENSOR CALIBRATION IN PROGRESS    ")
```

```
    print("-----")
```

```
    print("1. Calibrating Accelerometer (KEEP SENSOR FLAT & STILL)...")
```

```
    sx, sy, sz = 0, 0, 0
```

```
    valid = 0
```

```
    while valid < 100:
```

```
        vals = read_accel()
```

```
        if vals:
```

```
            sx += vals[0]
```

```
            sy += vals[1]
```

```
            sz += vals[2]
```

```
            valid += 1
```

```
            utime.sleep_ms(10)
```

```
    x_bias = sx / 100
```

```
    y_bias = sy / 100
```

```
    z_bias = (sz / 100) - 1024
```

```
    print("2. Calibrating Compass (ROTATE SENSOR IN AIR FOR 4 SEC)...")
```

```
    min_x, max_x = 32767, -32768
```

```
    min_y, max_y = 32767, -32768
```

```
    start_time = utime.ticks_ms()
```

```
while utime.ticks_diff(utime.ticks_ms(), start_time) < 4000:
```

```
    m = read_mag()
```

```
    if m:
```

```
        if m[0] < min_x: min_x = m[0]
```

```
        if m[0] > max_x: max_x = m[0]
```

```
        if m[1] < min_y: min_y = m[1]
```

```
        if m[1] > max_y: max_y = m[1]
```

```
    utime.sleep_ms(20)
```

```
mag_x_bias = (max_x + min_x) / 2
```

```
mag_y_bias = (max_y + min_y) / 2
```

```
print("Calibration Completed Successfully!\n")
```

```
# =====
```

3. AI MOTION MODEL & HEADING ALGORITHMS

```
# =====
```

```
def predict_activity_ai(magnitude, variance):
```

```
    """
```

```
    Evaluates motion features using decision tree thresholds.
```

```
    Locks step updates during non-walking motion.
```

```
    """
```

```
    if magnitude <= 18.6:
```

```
        return "SITTING"
```

```
    else:
```

```
        if variance > 275.5:
```

```
            return "SHAKING"
```

```
        else:
```

```
            return "WALKING"
```

```
def calculate_heading(mx, my):
```

```

mx_corr = mx - mag_x_bias
my_corr = my - mag_y_bias
heading_deg = math.degrees(math.atan2(my_corr, mx_corr))
if heading_deg < 0:
    heading_deg += 360.0
return heading_deg

```

```

def get_cardinal_direction(heading):
    if (heading >= 337.5) or (heading < 22.5): return "NORTH"
    elif 22.5 <= heading < 67.5: return "NORTH-EAST"
    elif 67.5 <= heading < 112.5: return "EAST"
    elif 112.5 <= heading < 157.5: return "SOUTH-EAST"
    elif 157.5 <= heading < 202.5: return "SOUTH"
    elif 202.5 <= heading < 247.5: return "SOUTH-WEST"
    elif 247.5 <= heading < 292.5: return "WEST"
    elif 292.5 <= heading < 337.5: return "NORTH-WEST"
    return "UNKNOWN"

```

```

# =====

```

```

# 4. MAIN GPS TRACKING LOOP

```

```

# =====

```

```

def main():
    global prev_x, prev_y, prev_z, filtered_mag, total_steps, total_distance, last_step_time,
    prev_activity

    global pos_x, pos_y, current_leg_dir, current_leg_dist

    init_lsm303dlhc()
    calibrate_sensors()

    print("=====")
    print("    LIVE PDR SENSOR GPS TELEMETRY ONLINE    ")

```

```
print("=====")
print("Starting Origin : X = 0.00 m, Y = 0.00 m")
print("Ready to track movement...")
print("-----")
```

```
while True:
```

```
    try:
```

```
        acc_vals = read_accel()
```

```
        mag_vals = read_mag()
```

```
        if acc_vals is None or mag_vals is None:
```

```
            continue
```

```
        x = acc_vals[0] - x_bias
```

```
        y = acc_vals[1] - y_bias
```

```
        z = acc_vals[2] - z_bias
```

```
        if prev_x is None:
```

```
            prev_x, prev_y, prev_z = x, y, z
```

```
            continue
```

```
        dx = x - prev_x
```

```
        dy = y - prev_y
```

```
        dz = z - prev_z
```

```
        prev_x, prev_y, prev_z = x, y, z
```

```
        raw_mag = math.sqrt(dx * dx + dy * dy + dz * dz)
```

```
        # Low-pass filter (EMA)
```

```
        if filtered_mag is None:
```

```
            filtered_mag = raw_mag
```

else:

filtered_mag = (ALPHA * raw_mag) + ((1 - ALPHA) * filtered_mag)

Sliding window variance calculation

mag_window.append(filtered_mag)

if len(mag_window) > 10:

mag_window.pop(0)

mean_mag = sum(mag_window) / len(mag_window)

variance = sum((m - mean_mag) ** 2 for m in mag_window) / len(mag_window)

current_time = utime.ticks_ms()

time_diff = utime.ticks_diff(current_time, last_step_time)

activity = predict_activity_ai(filtered_mag, variance)

heading = calculate_heading(mag_vals[0], mag_vals[1])

cardinal_dir = get_cardinal_direction(heading)

dist_to_start = math.sqrt(pos_x * pos_x + pos_y * pos_y)

STATE 1: WALKING (FULL GPS TELEMETRY UPDATE)

if activity == "WALKING":

if time_diff > STEP_COOLDOWN_MS:

last_step_time = current_time

total_steps += 1

total_distance += STRIDE_LENGTH_M

Trigonometric 2D position update

rad = math.radians(heading)


```

pos_x += STRIDE_LENGTH_M * math.sin(rad)
pos_y += STRIDE_LENGTH_M * math.cos(rad)

# Turn detection logic
if current_leg_dir is None:
    current_leg_dir = cardinal_dir
    current_leg_dist = STRIDE_LENGTH_M
elif cardinal_dir == current_leg_dir:
    current_leg_dist += STRIDE_LENGTH_M
else:
    route_history.append((current_leg_dir, current_leg_dist, heading))
    print("\n>>> TURN DETECTED <<<")
    print(f"Completed Leg : Walked {round(current_leg_dist, 2)} m towards
{current_leg_dir}")
    print(f"New Direction : Turning towards {cardinal_dir} ({round(heading, 1)}°)")
    print("-----")
    current_leg_dir = cardinal_dir
    current_leg_dist = STRIDE_LENGTH_M

# Print GPS Telemetry Block
print("\n=====")
print("    LIVE PDR SENSOR GPS TELEMETRY    ")
print("=====")
print("Current State : WALKING")
print(f"Total Distance: {round(total_distance, 2)} m | Total Steps: {total_steps}")
print(f"Heading    : {round(heading, 1)}° ({cardinal_dir}")
print(f"Coordinates  : X = {round(pos_x, 2)} m, Y = {round(pos_y, 2)} m")
print(f"Vector Motion : Moving {cardinal_dir} at {STRIDE_LENGTH_M} m/step")
print(f"Dist to Start : {round(dist_to_start, 2)} m")
print("=====")

```

```

# Loop closure detection

if total_steps > 8 and dist_to_start <= ORIGIN_RADIUS_M:

    print("\n=====")
    print("    ROUTE COMPLETED (RETURNED TO ORIGIN)    ")
    print("=====")
    print("Status      : LOOP CLOSURE CONFIRMED")
    print(f"Total Steps   : {total_steps} steps")
    print(f"Total Distance: {round(total_distance, 2)} m")
    print(f"Final Position: X = {round(pos_x, 2)} m, Y = {round(pos_y, 2)} m")
    print("\nJOURNEY ROUTE BREAKDOWN:")

    temp_history = route_history + [(current_leg_dir, current_leg_dist, heading)]
    for idx, (leg_dir, leg_dist, leg_head) in enumerate(temp_history, 1):
        print(f"  Leg {idx} : Walked {round(leg_dist, 2)} m towards {leg_dir} (Heading:
{round(leg_head, 1)}°)")

    print("=====\\n")

    prev_activity = "WALKING"

# -----

# STATE 2: SHAKING (TRACKING PAUSED)

# -----

elif activity == "SHAKING":

    if prev_activity != "SHAKING":

        print("\n>>> ACTION DETECTED <<<")
        print("Current State : SHAKING / UNSTABLE")
        print("GPS Tracking  : LOCKED (Distance & Coordinates Paused)")
        print(f"Coordinates   : X = {round(pos_x, 2)} m, Y = {round(pos_y, 2)} m")
        print("-----")

        prev_activity = "SHAKING"

# -----

```

```

# STATE 3: SITTING (TRACKING PAUSED)

# -----

elif activity == "SITTING":

    if prev_activity != "SITTING" and prev_activity is not None:

        print("\n>>> ACTION DETECTED <<<")

        print("Current State : SITTING / IDLE")

        print("GPS Tracking : LOCKED (Distance & Coordinates Paused)")

        print(f"Coordinates : X = {round(pos_x, 2)} m, Y = {round(pos_y, 2)} m")

        print("-----")

        prev_activity = "SITTING"

    utime.sleep_ms(30)

except KeyboardInterrupt:

    print("\nGPS Tracking Engine Stopped.")

    break

except Exception as e:

    print("Sensor Error:", e)

    utime.sleep_ms(200)

if __name__ == "__main__":

    main()

```

How to Run

1. Open **Thonny**, make sure your board is connected via USB, and open main.py.
2. Click **Run**.
3. Keep the sensor stationary on your desk for **Step 1** of calibration.
4. Rotate the sensor in a slow circular loop in the air for 4 seconds during **Step 2** (compass calibration).
5. Start moving or pushing the sensor along your path to see real-time updates!